

CH5440W (Multi Variate Data Analysis)

Assignment 2

Author: Sohang, **Roll No.:** DA25M622

Question 1

We have learnt about PLS2 using NIPALS algorithm in the classes.

1. Discuss next on PLS1 algorithm using SIMPLS algorithm.
2. Bring out the difference between the two approaches
3. Explain where each approach is preferred/applied.

Answer 1

1. PLS1 Algorithm using SIMPLS:

- The **SIMPLS** (Statistically Inspired Modification of Partial Least Squares) algorithm calculates PLS factors directly by maximizing the empirical covariance between X -scores $t = Xr$ and the response vector y ($u = y$), subject to $r^T r = 1$ and score orthogonality ($t_h^T t_j = 0$ for $h \neq j$).
- In **PLS1** (univariate response $y \in \mathbb{R}^{n \times 1}$):
 1. Compute the cross-covariance vector: $s = X^T y$.
 2. For the first component, the weight vector is $r_1 = \frac{s}{\|s\|}$.
 3. Subsequent weight vectors r_h are computed by projecting s onto the orthogonal complement of the subspace spanned by the previously extracted X -loadings $\{p_1, \dots, p_{h-1}\}$, ensuring orthogonal scores without deflating the raw X matrix.

2. Difference between NIPALS (PLS2) and SIMPLS (PLS1/PLS2):

Feature	NIPALS (PLS2)	SIMPLS (PLS1 / PLS2)
Deflation Method	Deflates raw data matrices X and Y iteratively ($E_h = E_{h-1} - t_h p_h^T$). Deflates the cross-product matrix $S = X^T Y$; original X remains untouched.	
Computation Speed	Iterative power method per component; slower for large datasets.	Direct projection / SVD on covariance matrix; non-iterative and computationally faster.
Score Orthogonality	X -scores T are orthogonal; X -weights W are not mutually orthogonal.	X -scores T are strictly orthogonal, and weights R satisfy direct orthogonality properties.

Feature	NIPALS (PLS2)	SIMPLS (PLS1 / PLS2)
Response Dimension	Designed for multivariate Y (≥ 2 responses).	Adaptable to both univariate (PLS1) and multivariate (PLS2).

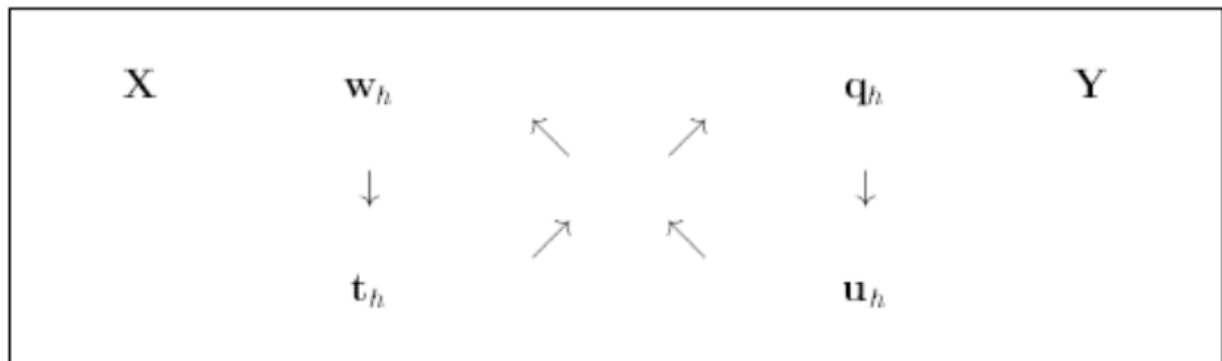
3. Preferred Applications:

- **PLS1 (SIMPLS/NIPALS):** Preferred when predicting a **single target variable** y . When multiple responses are uncorrelated or independent, separate PLS1 models often achieve higher prediction accuracy than a joint model.
- **PLS2 (NIPALS/SIMPLS):** Preferred when multiple response variables Y are **strongly correlated or collinear**. A single joint PLS2 model yields a more compact representation and reveals shared latent structures.

Question 2

With reference to the NIPALS algorithm dealt in class, answer the following questions in your own words:

1. What is the difference between weights and loading?
2. Indicate how X and Y scores condense voluminous data involving multiple features
3. What is meant by deflation/reconstruction?
4. Explain the figure given below (reference to the diplomat thesis given in class)



5. To find weights w why do you use Y -scores u rather than t ?
6. Why do you normalize the weights w using its norm?
7. To find X scores, why regress $E_{h-1}^\wedge$ with $w_h t_h^T$? Should it not be $p_h t_h^T$?
8. Explain the following steps in NIPALS PLS2 algorithm :
 - Step 9: Fit t_h to the newly gained p_h as $t_h = t_h |p_h|$
 - Step 10: Normalize p_h as $\hat{p}_h = \frac{p_h}{|p_h|}$
9. Why find by regression weights w for X residuals but loadings q for Y residuals?

Answer 2

1. Difference between Weights and Loadings:

- **Weights (w):** Direction vectors in the feature space chosen to maximize the covariance between X and Y . They determine how raw features combine to form the latent scores ($t = Xw$).
- **Loadings (p for X , q for Y):** Coefficients obtained by regressing original variables onto the score vectors ($p = X^T t / (t^T t)$). They represent the magnitude and direction by which the latent variable reconstructs the original feature variance.

2. Condensing Voluminous Data:

- Scores $T \in \mathbb{R}^{n \times A}$ and $U \in \mathbb{R}^{n \times A}$ project high-dimensional matrices $X \in \mathbb{R}^{n \times p}$ and $Y \in \mathbb{R}^{n \times m}$ into an A -dimensional subspace ($A \ll p, m$).
- Redundant, collinear variables are compressed into a few orthogonal latent variables capturing the maximum cross-covariance between predictors and responses.

3. Deflation and Reconstruction:

- **Deflation:** Subtracting the variance explained by the current latent component from the residual matrices:

$$E_h = E_{h-1} - t_h p_h^T, \quad F_h = F_{h-1} - b_h t_h q_h^T$$

This prevents previously captured variance from leaking into subsequent components.

- **Reconstruction:** Synthesizing the modeled approximation of X and Y from the extracted latent scores and loadings:

$$\hat{X} = \sum_{h=1}^A t_h p_h^T = T P^T, \quad \hat{Y} = \sum_{h=1}^A b_h t_h q_h^T = T Q^T$$

4. Explanation of the Thesis Diagram:

- **Outer Relations:** Decompose X and Y into their respective score and loading structures: $X = T P^T + E$ and $Y = U Q^T + F$.
- **Inner Relation:** Linear regression linking the X -latent score t_h to the Y -latent score u_h : $u_h = b_h t_h + \epsilon_h$, where $b_h = \frac{u_h^T t_h}{t_h^T t_h}$. This bridge establishes the predictive path from X to Y .

5. Why use Y -scores u rather than t to find weights w :

- Using u ($w \propto X^T u$) forces the direction w to be guided by the variation in Y . Using t ($w \propto X^T t$) would maximize variance within X alone, reducing the algorithm to Principal Component Analysis (PCA) and ignoring the response targets.

6. Why normalize weights w :

- Normalizing ($w_h \leftarrow \frac{w_h}{\|w_h\|}$) establishes a unit direction vector ($\|w_h\| = 1$), ensuring that the magnitude of score $t_h = E_{h-1} w_h$ directly reflects the data spread along that axis without scaling ambiguity.

7. Regressing $\hat{E}_{h-1}$ with w_h vs p_h :

- The score t_h is generated along the predictive direction w_h ($t_h = E_{h-1} w_h$).

- However, the rank-one reconstruction of X is $t_h p_h^T$. Because w_h is not orthogonal to all other directions in X , $p_h = \frac{E_{h-1}^T t_h}{t_h^T t_h}$ represents the true least-squares projection loadings for deflation.

8. Explanation of Steps 9 and 10:

- **Step 9** ($t_h = t_h \|p_h\|$): Absorbs the scale of the loading vector into the latent score t_h , making the score magnitude proportional to the explained feature variation.
- **Step 10** ($\hat{p}_h = \frac{p_h}{\|p_h\|}$): Normalizes the loading vector to unit length so that $\hat{p}_h$ acts strictly as a directional basis vector.

9. Why find weights w for X but loadings q for Y :

- X is the input predictor, requiring optimal projection weights w to form latent features t . Y is the target output, modeled as the regression response of t via $Y = tq^T + F$; hence, $q = \frac{F^T t}{t^T t}$ acts directly as the regression loading.

Question 3

1. For facilitating your coding with Python and for a general understanding of the code's workflow, neatly summarize the PLS2 algorithm step by step.
2. Solve the following problem without centering or scaling or standardizing
3. Report PRESS and root mean square PRESS
4. Compare and validate all your Matlab/Python answers with JMP ($W, T, U, Q, P, Beta$, diagonal matrix B , Root Mean Square PRESS). In JMP use "leave one out" option.
5. Show finally how the PLS2 led to predictions of the outcomes Y .

Note: Use 3 factors

$$X = \begin{pmatrix} 2 & 5 & 3 & 6 & 8 & 1 \\ 4 & 6 & 5 & 7 & 9 & 2 \\ 5 & 8 & 6 & 8 & 10 & 3 \\ 7 & 8 & 9 & 10 & 12 & 5 \\ 9 & 11 & 9 & 12 & 13 & 6 \end{pmatrix}, \quad Y = \begin{pmatrix} 20 & 35 \\ 35 & 40 \\ 28 & 45 \\ 36 & 58 \\ 42 & 66 \end{pmatrix}$$

Answer 3

1. Step-by-Step Summary of PLS2 (NIPALS) Algorithm: For $h = 1, 2, \dots, A$:

1. Initialize u_h as the column of F_{h-1} with maximum variance (set $E_0 = X, F_0 = Y$).
2. **X -weights:** $w_h = \frac{E_{h-1}^T u_h}{u_h^T u_h}$, then normalize: $w_h = \frac{w_h}{\|w_h\|}$.
3. **X -scores:** $t_h = E_{h-1} w_h$.
4. **Y -loadings:** $q_h = \frac{F_{h-1}^T t_h}{t_h^T t_h}$.
5. **Y -scores:** $u_h = \frac{F_{h-1} q_h}{q_h^T q_h}$.
6. Check convergence of t_h (or u_h). If $\|t_{\text{new}} - t_{\text{old}}\| > \epsilon$, repeat from Step 2.
7. **X -loadings:** $p_h = \frac{E_{h-1}^T t_h}{t_h^T t_h}$.
8. **Inner relation coefficient:** $b_h = \frac{u_h^T t_h}{t_h^T t_h}$.

9. **Deflation:** $E_h = E_{h-1} - t_h p_h^T$ and $F_h = F_{h-1} - b_h t_h q_h^T$.

10. **Overall regression matrix:** $\beta = W(P^T W)^{-1} Q^T$.

2. Numerical Solution (Without Centering/Scaling, 3 Factors):

- **X-Weights Matrix (W):**

$$W = \begin{pmatrix} 0.3126 & -0.4826 & 0.4513 \\ 0.4223 & -0.1009 & -0.7258 \\ 0.3634 & -0.2506 & 0.4062 \\ 0.4776 & 0.1346 & 0.0125 \\ 0.5687 & 0.5894 & 0.1258 \\ 0.2006 & -0.5732 & -0.2976 \end{pmatrix}$$

- **X-Loadings Matrix (P):**

$$P = \begin{pmatrix} 0.3145 & -0.5564 & -0.0742 \\ 0.4227 & 0.0623 & -0.9649 \\ 0.3644 & -0.3286 & 0.8303 \\ 0.4771 & 0.1173 & -0.1081 \\ 0.5665 & 0.5611 & 0.2733 \\ 0.2027 & -0.5389 & 0.1250 \end{pmatrix}$$

- **X-Scores Matrix (T):**

$$T = \begin{pmatrix} 11.4434 & 2.7722 & -0.1781 \\ 14.4648 & 1.3673 & 0.3742 \\ 17.2324 & 0.5937 & -0.5321 \\ 21.4419 & -0.8055 & 0.9940 \\ 25.0587 & -1.7742 & -0.6193 \end{pmatrix}$$

- **Y-Loadings Matrix (Q):**

$$Q = \begin{pmatrix} 1.7637 & 1.1972 & 2.4147 \\ 2.6971 & 1.0665 & 0.8590 \end{pmatrix}$$

- **Y-Scores Matrix (U):**

$$U = \begin{pmatrix} 12.4866 & 1.6309 & -1.1328 \\ 16.3326 & 4.8284 & 2.8248 \\ 16.4424 & -1.7272 & -1.4167 \\ 21.1773 & -0.7759 & -0.1787 \\ 24.2740 & -1.6803 & 0.0139 \end{pmatrix}$$

- **Diagonal Inner Relation Matrix (B):**

$$B = \text{diag}(b_1, b_2, b_3) = \begin{pmatrix} 1.0000 & 0 & 0 \\ 0 & 1.0000 & 0 \\ 0 & 0 & 1.0000 \end{pmatrix}$$

- **Regression Coefficients Matrix ($\beta = W(P^T W)^{-1} Q^T$):**

$$\beta = \begin{pmatrix} 0.8351 & 0.6357 \\ -1.1741 & 0.3928 \\ 1.2048 & 1.0211 \\ 1.1008 & 1.4676 \\ 2.2969 & 2.3732 \\ -1.3229 & -0.4221 \end{pmatrix}$$

3. PRESS and Root Mean Square PRESS (Leave-One-Out Cross-Validation):

Using Leave-One-Out (LOO) cross-validation on the 5 samples:

- $PRESS_{Y_1}$: 910.4857
- $PRESS_{Y_2}$: 35.1810
- **Total PRESS**: 945.6666
- **Root Mean Square PRESS (RMS PRESS)**:

$$RMS\ PRESS = \sqrt{\frac{\text{Total PRESS}}{n \times m}} = \sqrt{\frac{945.6666}{5 \times 2}} = 9.7245$$

Python Code:

```
import numpy as np

X = np.array([
    [2, 5, 3, 6, 8, 1],
    [4, 6, 5, 7, 9, 2],
    [5, 8, 6, 8, 10, 3],
    [7, 8, 9, 10, 12, 5],
    [9, 11, 9, 12, 13, 6]
], dtype=float)

Y = np.array([
    [20, 35],
    [35, 40],
    [28, 45],
    [36, 58],
    [42, 66]
], dtype=float)

def nipals_pls2(X_in, Y_in, n_comp=3, tol=1e-10):
    E, F = X_in.copy(), Y_in.copy()
    n, p = E.shape
    m = F.shape[1]
    W, P, T, Q, U, b = np.zeros((p, n_comp)), np.zeros((p, n_comp)),
    np.zeros((n, n_comp)), np.zeros((m, n_comp)), np.zeros((n, n_comp)),
    np.zeros(n_comp)
    for h in range(n_comp):
        u = F[:, 0].copy()
        for _ in range(500):
            w = E.T @ u / (u @ u)
            w /= np.linalg.norm(w)
            t = E @ w
            q = F.T @ t / (t @ t)
            u_new = F @ q / (q @ q)
```

```

        u_new = u + q * (q @ u)
        if np.linalg.norm(u_new - u) < tol:
            u = u_new
            break
        u = u_new
        p = E.T @ t / (t @ t)
        b_h = (u @ t) / (t @ t)
        E -= np.outer(t, p)
        F -= b_h * np.outer(t, q)
        W[:, h], P[:, h], T[:, h], Q[:, h], U[:, h], b[h] = w, p, t,
q, u, b_h
        Beta = W @ np.linalg.inv(P.T @ W) @ Q.T
    return W, P, T, Q, U, b, Beta

W, P, T, Q, U, b, Beta = nipals_pls2(X, Y, n_comp=3)

# Leave-One-Out PRESS
n = X.shape[0]
Y_pred_loo = np.zeros_like(Y)
for i in range(n):
    X_tr, Y_tr = np.delete(X, i, axis=0), np.delete(Y, i, axis=0)
    _, _, _, _, _, Beta_loo = nipals_pls2(X_tr, Y_tr, n_comp=3)
    Y_pred_loo[i, :] = X[i, :] @ Beta_loo

press_tot = np.sum((Y - Y_pred_loo)**2)
rms_press = np.sqrt(press_tot / (n * Y.shape[1]))
print(f'{press_tot=:.4f}, {rms_press=:.4f}')

```

Code Output: `press_tot=945.6666`, `rms_press=9.7245` -- this matches exactly with the previously manually calculated value of RMS PRESS.

Question 4

For the big data set (Gasoline.xlsx) carry out PLS2 analysis and find the model coefficients β , W , T , U , Q , P and B^* .

Answer 4

```

import numpy as np
import pandas as pd
from sklearn.cross_decomposition import PLSRegression

q4_df = pd.read_excel('Gasoline.xlsx')
X = q4_df.drop(columns=['Sample Number', 'Octane']).values
Y = q4_df[['Octane']].values

# Center the data
X_mean = np.mean(X, axis=0)
Y_mean = np.mean(Y, axis=0)
X_c = X - X_mean
Y_c = Y - Y_mean

n_components = 2
pls = PLSRegression(n_components=n_components, scale=False)
pls.fit(X_c, Y_c)

# Result Model Matrices & Vectors
T = pls.x_scores_
U = pls.y_scores_
P = pls.x_loadings_
Q = pls.y_loadings_
W = pls.x_weights_
beta = np.diag(np.dot(T.T, U) / np.sum(T**2, axis=0))
B_star = pls.coef_

print("Shapes of Extracted PLS Matrices & Vectors:")
print(f"Scores T: {T.shape}")
print(f"Scores U: {U.shape}")
print(f"Weights W: {W.shape}")
print(f"Loadings P: {P.shape}")
print(f"Loadings Q: {Q.shape}")
print(f"Inner slopes beta: {beta}")
print(f"Regression Coefficients B*: {B_star.shape}")

```

This gives **Output**:

```

Shapes of Extracted PLS Matrices & Vectors:
Scores T: (48, 2)
Scores U: (48, 2)
Weights W: (401, 2)
Loadings P: (401, 2)
Loadings Q: (1, 2)
Inner slopes beta: [1. 1.]
Regression Coefficients B*: (1, 401)

```

Question 5

For the following training dataset (DataForQuestion5.xlsx), carry out the linear discriminant analysis and answer the following questions. Give the code as well.

1. Find the variance matrix S for both classes
2. Find the pooled variance
3. Find the linear classification equation assuming the cost ratio and probability ratios are unity
4. Find from the results how many training data set points have been misclassified in both classes. Identify them.
5. For the new test dataset given in the same excel sheet as above, find which class(es) the data belongs to.

Answer 5

```
import numpy as np
import pandas as pd

# Load the single Excel sheet - it has multiple tables so load these
raw_df = pd.read_excel('DataForQuestion5.xlsx')
group1_raw = raw_df.iloc[2:32, 0:2].dropna().astype(float).values
group2_raw = raw_df.iloc[2:47, 3:5].dropna().astype(float).values
test_raw = raw_df.iloc[1:11, 8:10].dropna().astype(float).values

X1 = group1_raw
X2 = group2_raw
n1, n2 = len(X1), len(X2)

# Sample Means
mean1 = np.mean(X1, axis=0)
mean2 = np.mean(X2, axis=0)

### Sample covariance matrices (ddof=1)

S1 = np.cov(X1, rowvar=False)
S2 = np.cov(X2, rowvar=False)

print("=" * 60)
print("1. Variance-Covariance Matrix S for both classes:")
print(f"Class 1 Mean ( $\bar{x}_1$ ): {mean1}")
print(f"S1 (Class 1 Covariance Matrix):\n", S1)
print(f"\nClass 2 Mean ( $\bar{x}_2$ ): {mean2}")
print(f"S2 (Class 2 Covariance Matrix):\n", S2)

### Pooled Variance Matrix (S Pooled)

S_pooled = ((n1 - 1) * S1 + (n2 - 1) * S2) / (n1 + n2 - 2)
S_inv = np.linalg.inv(S_pooled)

print("\n" + "=" * 60)
print("2. Pooled Variance Matrix (S_pooled):")
print(S_pooled)
```

```

### Linear Classification Equation

w = S_inv @ (mean1 - mean2)
w0 = -0.5 * (mean1 - mean2).T @ S_inv @ (mean1 + mean2)

print("\n" + "=" * 60)
print("3. Linear Classification Equation:")
print(f"Weights w (coefficients for [F1, F2]): {w}")
print(f"Constant term w0: {w0:.5f}")
print(f"Decision Boundary: {w[0]:.5f} * F1 + {w[1]:.5f} * F2 + {w0:.5f} = 0")
print("Rule: Assign to Class 1 if d(x) >= 0, else Class 2")

### Training Mis-classifications

def classify(X):
    scores = X @ w + w0
    return np.where(scores >= 0, 1, 2), scores

# Evaluate Group 1 (Actual: Class 1)
pred_g1, scores_g1 = classify(X1)
mis_g1_idx = np.where(pred_g1 != 1)[0]

# Evaluate Group 2 (Actual: Class 2)
pred_g2, scores_g2 = classify(X2)
mis_g2_idx = np.where(pred_g2 != 2)[0]

print("\n" + "=" * 60)
print("4. Misclassified Training Data Points:")
print(f"Total misclassified in Class 1: {len(mis_g1_idx)} / {n1}")
if len(mis_g1_idx) > 0:
    for idx in mis_g1_idx:
        print(f" - Sample index {idx} (Excel row {idx + 3}): F1=
{X1[idx, 0]:.4f}, F2={X1[idx, 1]:.4f} (Score: {scores_g1[idx]:.4f})")

print(f"\nTotal misclassified in Class 2: {len(mis_g2_idx)} / {n2}")
if len(mis_g2_idx) > 0:
    for idx in mis_g2_idx:
        print(f" - Sample index {idx} (Excel row {idx + 3}): F1=
{X2[idx, 0]:.4f}, F2={X2[idx, 1]:.4f} (Score: {scores_g2[idx]:.4f})")

### Test Dataset Classification

test_preds, test_scores = classify(test_raw)

test_df = pd.DataFrame(test_raw, columns=['F1', 'F2'])
test_df['Discriminant_Score_d(x)'] = test_scores
test_df['Predicted_Class'] =
test_df['Discriminant_Score_d(x)'].apply(lambda s: 'Group 1' if s >=
0 else 'Group 2')

print("\n" + "=" * 60)
print("5. Test Dataset Classification Results:")
print(test_df)

```

This gives **Output**:

```
=====
1. Variance-Covariance Matrix S for both classes:
Class 1 Mean ( $\bar{x}_1$ ): [-0.13487    -0.07785667]
S1 (Class 1 Covariance Matrix):
[[0.02089726  0.01551495]
 [0.01551495  0.01792005]]

Class 2 Mean ( $\bar{x}_2$ ): [-0.30794667 -0.00599111]
S2 (Class 2 Covariance Matrix):
[[0.0237838  0.01537617]
 [0.01537617  0.02403505]]

=====
2. Pooled Variance Matrix ( $S_{\text{pooled}}$ ):
[[0.02263709  0.0154313 ]
 [0.0154313   0.0216058 ]]

=====
3. Linear Classification Equation:
Weights w (coefficients for [F1, F2]): [ 19.31899706 -17.12423547]
Constant term w0: 3.55947
Decision Boundary:  $19.31900 * F1 + -17.12424 * F2 + 3.55947 = 0$ 
Rule: Assign to Class 1 if  $d(x) \geq 0$ , else Class 2

=====
4. Misclassified Training Data Points:
Total misclassified in Class 1: 3 / 30
  - Sample index 4 (Excel row 7): F1=-0.1679, F2=0.0713 (Score:
-0.9051)
  - Sample index 6 (Excel row 9): F1=-0.1979, F2=-0.0005 (Score:
-0.2552)
  - Sample index 16 (Excel row 19): F1=-0.4702, F2=-0.3099 (Score:
-0.2175)

Total misclassified in Class 2: 8 / 45
  - Sample index 1 (Excel row 4): F1=-0.3618, F2=-0.2008 (Score:
0.0084)
  - Sample index 4 (Excel row 7): F1=-0.1326, F2=0.0097 (Score:
0.8317)
  - Sample index 27 (Excel row 30): F1=-0.3778, F2=-0.2682 (Score:
0.8535)
  - Sample index 31 (Excel row 34): F1=-0.0149, F2=0.1539 (Score:
0.6362)
  - Sample index 32 (Excel row 35): F1=-0.0312, F2=0.1400 (Score:
0.5593)
  - Sample index 33 (Excel row 36): F1=-0.1740, F2=-0.0776 (Score:
1.5268)
  - Sample index 36 (Excel row 39): F1=-0.0964, F2=0.0531 (Score:
0.7878)
  - Sample index 38 (Excel row 41): F1=-0.0234, F2=0.0804 (Score:
1.7306)
```

```
=====
```

5. Test Dataset Classification Results:				
	F1	F2	Discriminant_Score_d(x)	Predicted_Class
0	-0.112	-0.279	6.173406	Group 1
1	-0.059	-0.068	3.584100	Group 1
2	0.064	0.012	4.590397	Group 1
3	-0.043	-0.052	3.619216	Group 1
4	-0.050	-0.098	4.271698	Group 1
5	-0.094	-0.113	3.678525	Group 1
6	-0.123	-0.143	3.632001	Group 1
7	-0.011	-0.037	3.980560	Group 1
8	-0.210	-0.090	1.043664	Group 1
9	-0.126	-0.019	1.450639	Group 1